Deep Generative Models: VAE vs DDPM

GenCV003 — Implementation Report

Abstract

This report covers the from-scratch implementation, training, and evaluation of two generative models: a Variational Autoencoder (VAE) and a Denoising Diffusion Probabilistic Model (DDPM). Both are trained on standard benchmarks and evaluated using FID and Inception Score via a pretrained InceptionV3 network.

Part 1 — Variational Autoencoder (VAE)

1.1 Theoretical Background

A VAE is a generative model that learns a compressed representation of data. It has two parts: an encoder that maps an input image to a latent vector, and a decoder that reconstructs an image from that vector. The model is trained to balance two objectives simultaneously:

- **Reconstruct the input accurately** — the decoded image should resemble the original.
- **Keep the latent space organized** — the encoder's distribution should stay close to a standard normal $\mathcal{N}(0, 1)$, measured by the KL divergence.

The natural goal is to maximize the log-likelihood of the data:

$$\log p(x) = \log \int p(x|z)\, p(z)\, dz$$

This integral over all possible z is intractable — it cannot be computed in closed form.

Paper ref — Section 1: "The marginal likelihood $p(x)$ is intractable . . . and we cannot use the EM algorithm."

The ELBO — What We Actually Optimize

Since $\log p(x)$ is intractable, the paper derives a lower bound called the Evidence Lower BOund (ELBO):

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction term}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z|x) \,\|\, p(z))}_{\text{regularisation term}}$$

Paper ref — Section 2, Equation 1 (ELBO derivation); Section 3 (Variational Autoencoder).

The Reparameterization Trick

To train the encoder via backpropagation, gradients must flow through the sampling step $z \sim q(z|x)$. But sampling is not differentiable. The solution is to rewrite the sample as a deterministic function of the parameters and external noise:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

The randomness (ε) is separated from the learnable parameters (μ, σ). ε is treated as an external input — no gradient is needed through it. Gradients flow cleanly through μ and σ. Without this trick, the model cannot be trained end-to-end with standard backpropagation, as illustrated in Figure 1.

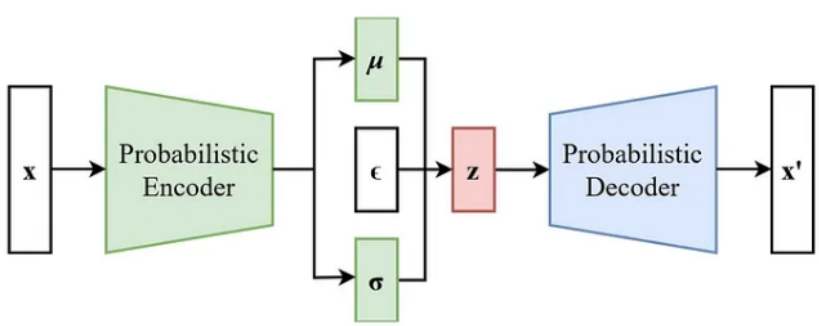


Figure 1: VAE architecture

1.2 Implementation

Two VAE variants were implemented: an MLP-based model following the original paper, and a convolutional model designed for CIFAR-10 to enable a fair comparison with DDPM.

VAE-MLP

The MLP VAE follows Appendix C.1 of Kingma & Welling (2013) exactly, trained on MNIST (28×28 grayscale, input dimension 784). The full architecture is shown in Figure 2.

- **Encoder:** Input $\rightarrow$ FC(400) $\rightarrow$ ReLU $\rightarrow$ FC(200) $\rightarrow$ ReLU $\rightarrow$ separate linear heads for μ and $\log \sigma^2$
- **Decoder:** $z \rightarrow$ FC(200) $\rightarrow$ ReLU $\rightarrow$ FC(400) $\rightarrow$ ReLU $\rightarrow$ FC(input_size) $\rightarrow$ Sigmoid

Paper ref — Appendix C.1 (MNIST architecture)

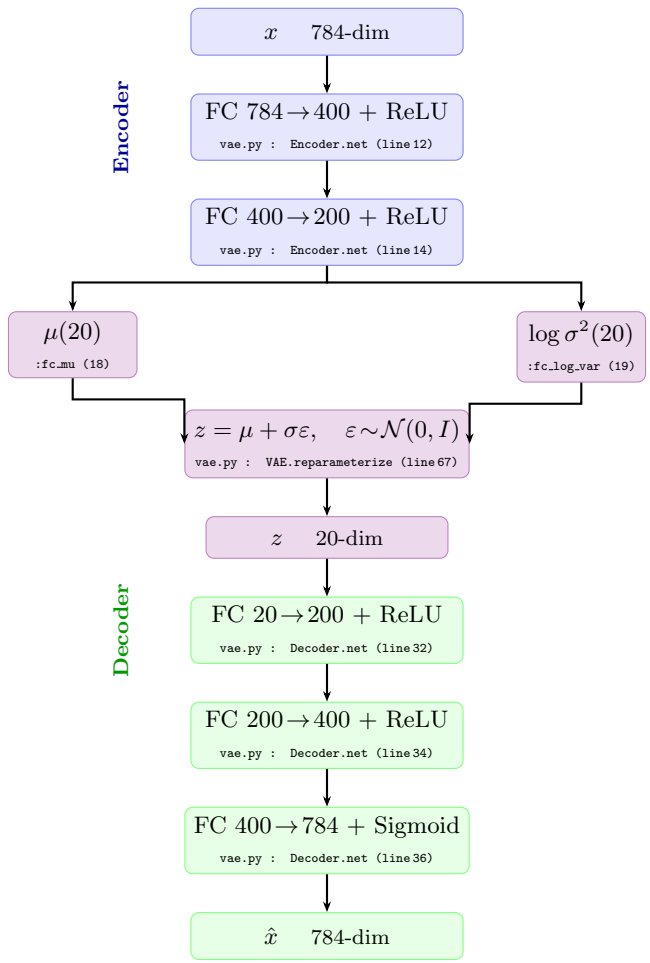


Figure 2: MLP-VAE architecture (`models/vae.py`). Encoder maps x to Gaussian parameters $(\mu, \log \sigma^2)$ via `Encoder` (line 6); the reparameterization trick in `VAE.reparameterize` (line 67) samples $z = \mu + \sigma\varepsilon$ while keeping gradients flowing; the decoder in `Decoder` (line 26) reconstructs $\hat{x}$.

VAE-CONV

For a fair comparison with DDPM — which operates on CIFAR-10 (32×32 RGB) — a convolutional VAE was implemented on the same dataset and data normalisation ($[-1, 1]$). Using an MLP on 32×32 RGB (3,072 dims) would be prohibitively large. The architecture uses strided convolutions for downsampling (encoder) and transposed convolutions for upsampling (decoder), as shown in Figure 3.

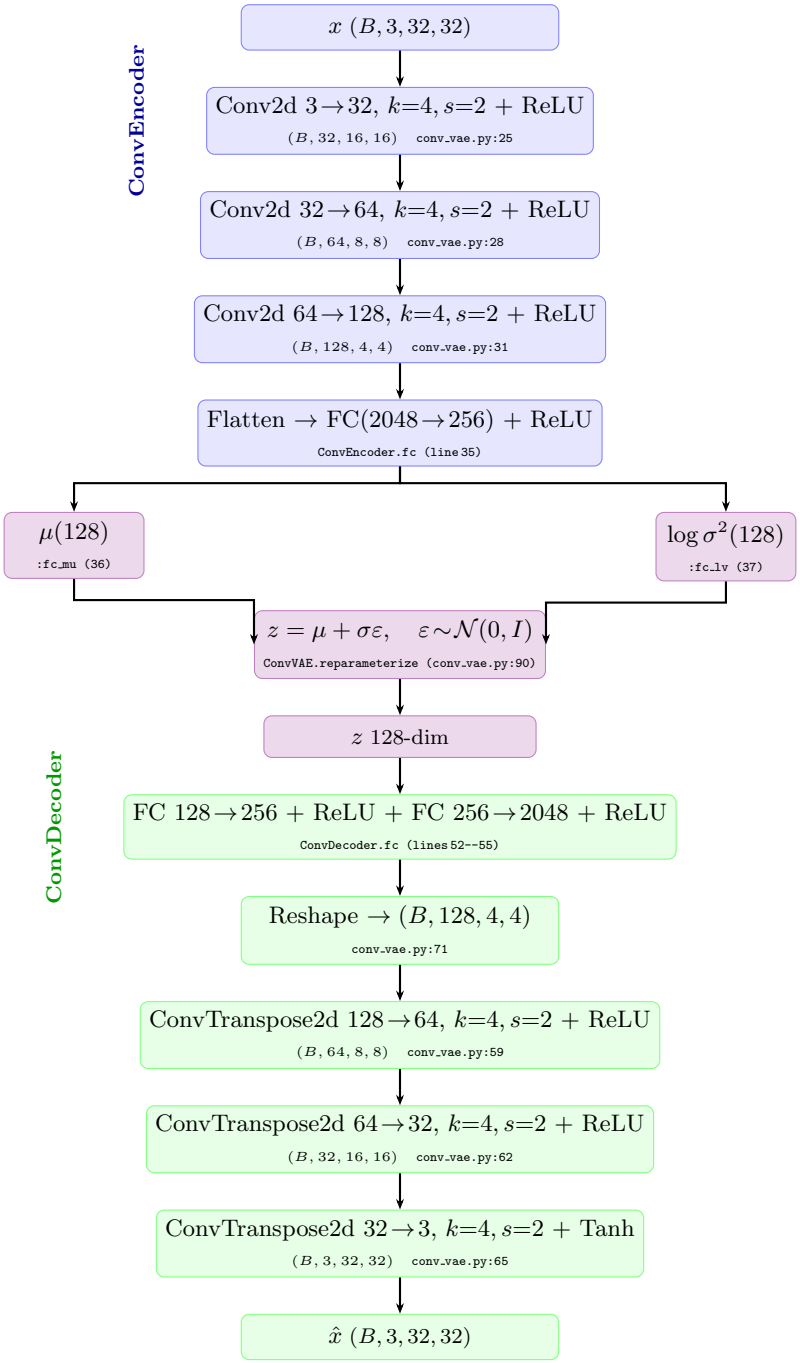


Figure 3: ConvVAE architecture (`models/conv_vae.py`). The encoder (`ConvEncoder`, line 19) uses three stride-2 Conv2d layers to compress $(B, 3, 32, 32)$ to $(B, 128, 4, 4)$, then a FC layer produces μ and $\log \sigma^2$. Reparameterization (`ConvVAE.reparameterize`, line 90) samples $z \in \mathbb{R}^{128}$. The decoder (`ConvDecoder`, line 46) mirrors the encoder with transposed convolutions, outputting $\hat{x} \in [-1, 1]$ via Tanh.

KL Divergence — Closed Form

For Gaussian encoder $q(z|x) = \mathcal{N}(\mu, \sigma^2 I)$ and prior $p(z) = \mathcal{N}(0, I)$, the KL divergence has a closed form — no sampling needed:

$$D_{\mathrm{KL}} = -\frac{1}{2} \sum_{j=1}^{J} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

Paper ref — Appendix B, Equation B.3.

Reconstruction Loss — Two Variants

- **MNIST:** pixel values are binary → $p(x|z)$ modeled as Bernoulli → Binary Cross-Entropy loss
- **Frey Face:** pixel values are continuous → $p(x|z)$ modeled as Gaussian → MSE loss

Paper ref — Appendix C.1 (Bernoulli MLP for MNIST), Appendix C.2 (Gaussian MLP for Frey Face).

1.3 Training Configuration

Dataset 1 — MNIST

MNIST contains 60,000 grayscale handwritten digit images (28×28 pixels). Since pixel values represent ink intensity in $[0, 1]$, they are treated as Bernoulli probabilities $\rightarrow$ BCE loss.

Table 1: VAE-MLP Training Settings (MNIST)

Setting	**Value**	**Paper source**
Input dimension	784 (28×28)	Appendix C.1
Latent dimension	20	Appendix C.1: "$L = 20$"
Hidden dimension	400	Appendix C.1 (200 in paper; 400 used here)
Reconstruction loss	BCE (Bernoulli)	Appendix C.1
Optimizer	Adam, lr=10^{-3}	Section 5 / Appendix C
Batch size	128	Appendix C
Epochs	50	—

Paper ref — Appendix C.1: "We used . . . a Bernoulli MLP as decoder."

1.4 Results

The VAE-MLP is evaluated separately to show the concept.

MNIST — Reconstructions

The model learns to reconstruct digits cleanly within a few epochs. By epoch 10 the outputs are recognizable; by epoch 50 they are accurate, as shown in Figure 4.

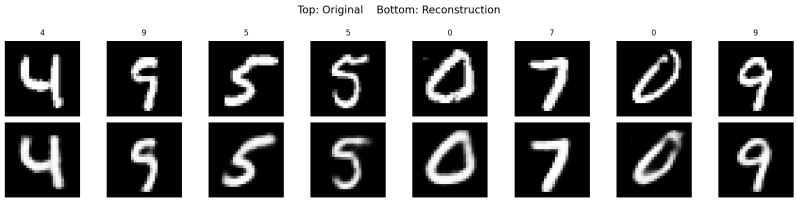


Figure 4: VAE reconstructions on MNIST (top: original, bottom: reconstructed)

MNIST — Generated Samples

Sampling by drawing $z \sim \mathcal{N}(0, I)$ and decoding produces plausible digits, as shown in Figure 5. Samples are somewhat blurry compared to real images — a known limitation of VAEs. The reconstruction loss favors pixel-level accuracy over sharpness.

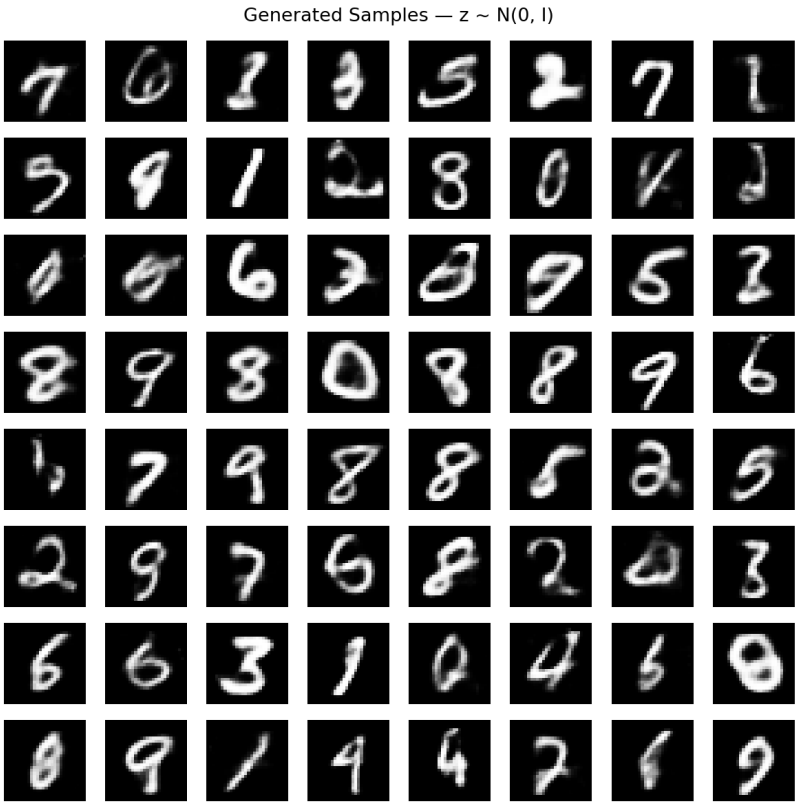


Figure 5: Random samples generated by the VAE from MNIST latent space

MNIST — Latent Space Interpolation

Linearly interpolating between two points in latent space produces smooth transitions between digits, as shown in Figure 6. This demonstrates the KL regularization working as intended — the latent space is continuous and meaningful.

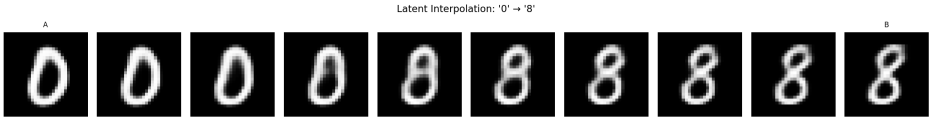


Figure 6: Smooth interpolation between digit pairs in the 20-dim latent space

Paper ref — Section 3: "The KL term encourages the encoder to be close to the prior, making the latent space . . . "

Part 2 — Denoising Diffusion Probabilistic Model (DDPM)

2.1 Theoretical Background

A DDPM takes a completely different approach to generative modeling. Instead of learning a compressed representation, it learns to reverse a gradual noising process. The model defines two paired Markov chains: a fixed forward process q that gradually adds noise to data, and a learned reverse process p_θ that denoises, as illustrated in Figure 7.

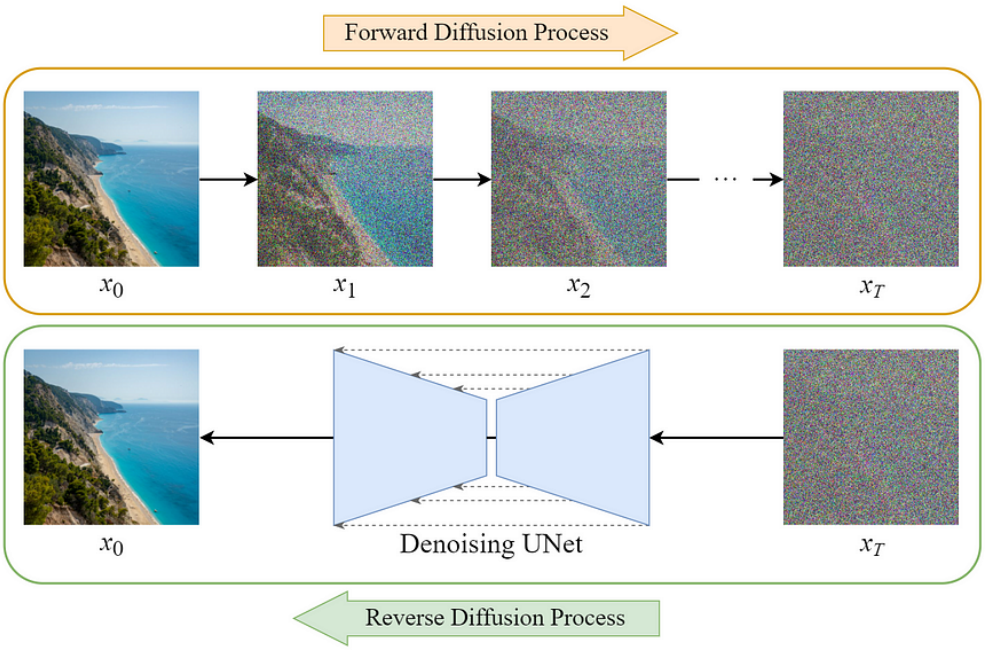


Figure 7: Overview of the diffusion model: forward process q (left→right) and reverse process p_θ (right→left)

2.2 Forward Process

The forward process q gradually corrupts a real image by adding small amounts of Gaussian noise at each of $T = 1000$ timesteps until the image becomes pure noise. The variance schedule is fixed — it is not learned. Using the reparameterization with $\bar{\alpha}_t = \prod \alpha_s$ (cumulative product), any timestep t can be reached in one shot without iterating step by step, as shown in Figure 8:

$$q(x_t|x_0) = \mathcal{N}(x_t;\ \sqrt{\bar{\alpha}_t}\, x_0,\ (1 - \bar{\alpha}_t)\, I)$$

Paper ref — Section 2, Equations 2 and 4. This closed form is what makes training efficient.

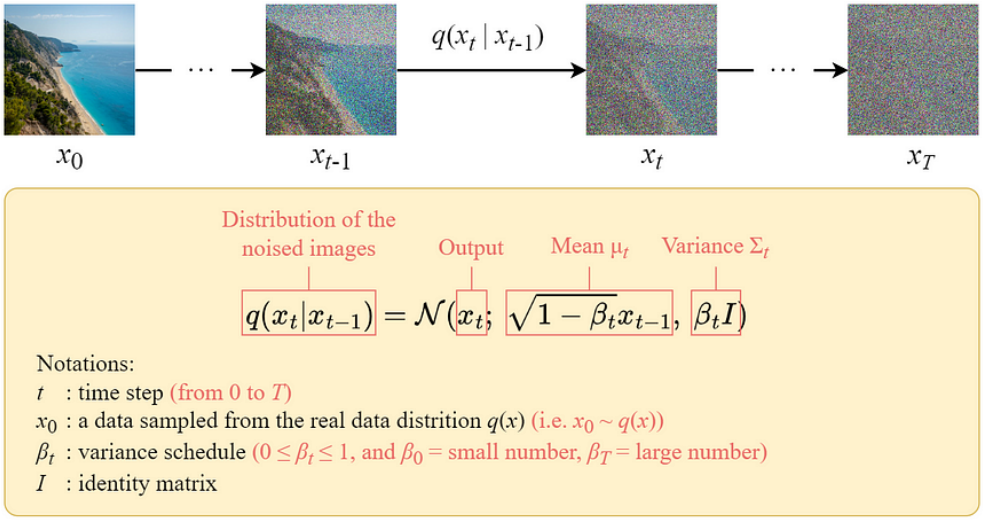


Figure 8: Forward diffusion process: clean image x_0 is gradually corrupted over $T = 1000$ steps

Representative training examples used during the CIFAR-10 run are shown in Figure 9.

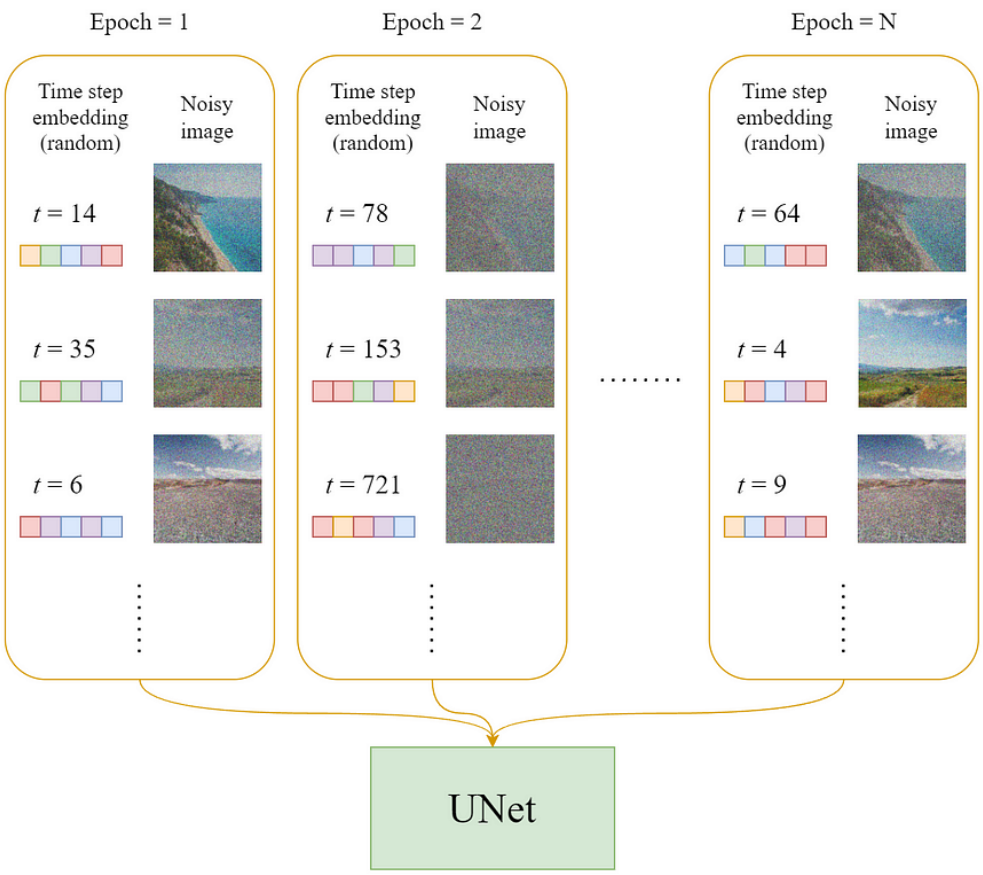


Figure 9: Examples of the training data used for the diffusion model

The noise schedule controls how quickly the image is corrupted. A linear schedule is used, increasing β_t from a very small value at $t = 1$ to 0.02 at $t = T$.

Paper ref — Section 4: "We set $T = 1000$ ... forward process variances to constants increasing linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$."

2.3 Model Architecture (U-Net)

The reverse process is parameterized by a U-Net that takes a noisy image x_t and timestep t as input, and predicts the noise ε that was added. The U-Net has four resolution levels ($32\times32 \to 16\times16 \to 8\times8 \to 4\times4$) with skip connections, as shown in Figure 10.

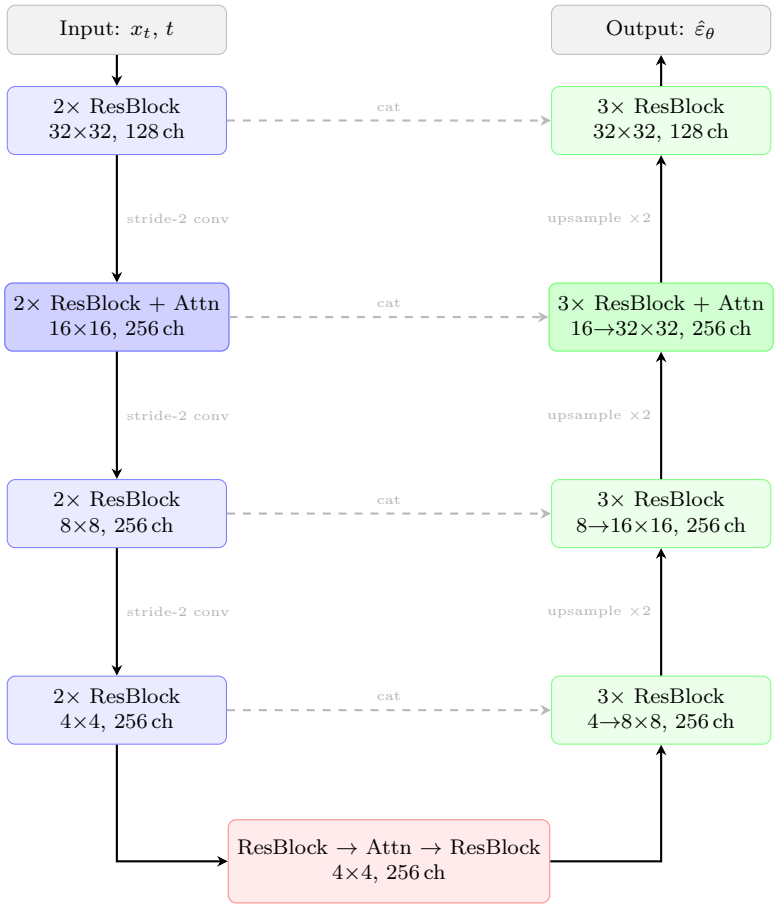


Figure 10: U-Net backbone $\varepsilon_\theta(x_t, t)$ (`models/ddpm.py : UNet`). Blue: encoder, downsamples via stride-2 conv. Red: bottleneck. Green: decoder, upsamples via nearest-neighbour + conv. Dashed arrows: skip connections (channel-wise concatenation). Darker shading marks levels with self-attention (16×16 resolution only). Implementation: `UNet.__init__` (line 171), `UNet.forward` (line 264).

Paper ref — Appendix B: "Our neural network architecture follows the backbone of PixelCNN++ . . . a U-Net based on a Wide ResNet. Our 32×32 models use four feature map resolutions (. . .) and all models have two convolutional residual blocks per resolution level."

Component 1 — Time Embedding

The scalar timestep t must be communicated to the network so every block knows which noise level it is working with. A sinusoidal positional embedding (borrowed from Transformers) encodes t into a rich 128-dim vector, which is then projected to 256 dims and injected into every ResBlock, as shown in Figure 11.

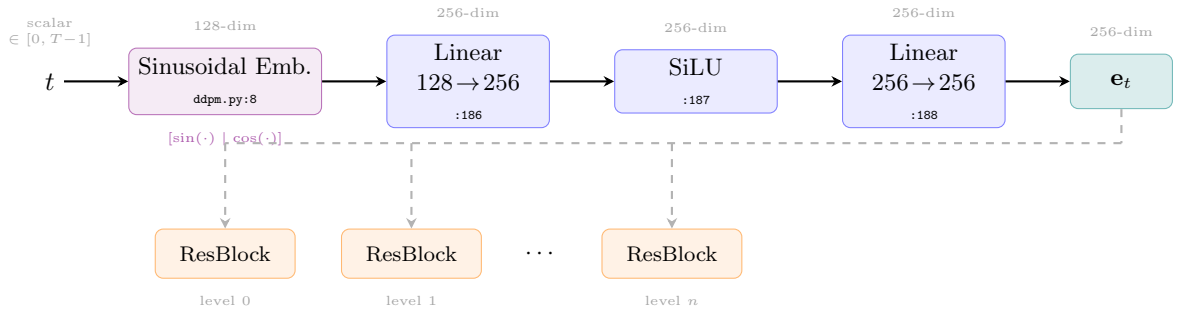


Figure 11: Time embedding (`ddpm.py:8`, `time_mlp:184`). Scalar t is sinusoidally embedded to 128-dim, projected by two FC layers to 256-dim $\mathbf{e}_t$, and broadcast into every ResBlock.

Paper ref — Appendix B: "Diffusion time t is specified by adding the Transformer sinusoidal position embedding [60] into each residual block." Reference [60] is Vaswani et al. (2017).

Component 2 — Residual Block

The residual block is the core building unit of the U-Net. It applies two convolutions with a skip connection, and injects the time embedding between the two convolutions so every block is aware of the current noise level, as illustrated in Figure 12.

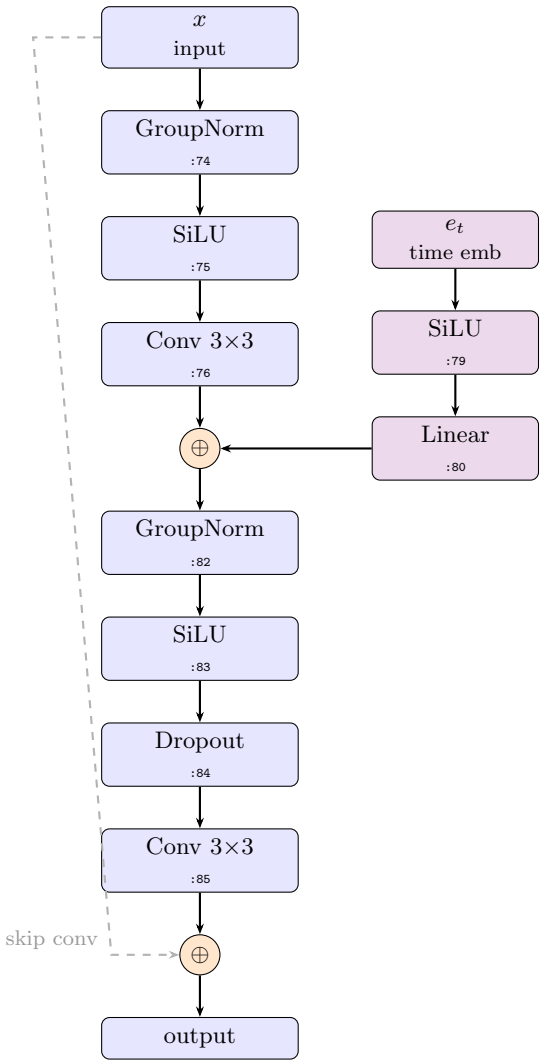


Figure 12: Residual block data flow (`models/ddpm.py : ResidualBlock`, line 48). Time embedding e_t is injected via `time_act`+`time_linear` (lines 79–80) between the two Conv 3×3 layers. Skip 1×1 conv (lines 88–92) bypasses the block when channel counts differ. Line numbers reference `ResidualBlock.forward` (line 94).

- GroupNorm replaces weight normalization for better training stability.
- SiLU activation provides smooth gradient flow.
- Dropout rate = 0.1 applied to the second convolution.
- 1×1 skip conv when input/output channels differ; Identity otherwise.

Paper ref — Appendix B: "We replaced weight normalization [49] with group normalization [66]." Section 4: "We set the dropout rate on CIFAR10 to 0.1."

Component 3 — Self-Attention Block

Self-attention is inserted at the 16×16 resolution only. It lets the network look at the entire feature map at once and capture long-range spatial structure — important for generating images where distant regions must be globally consistent, as shown in Figure 13.

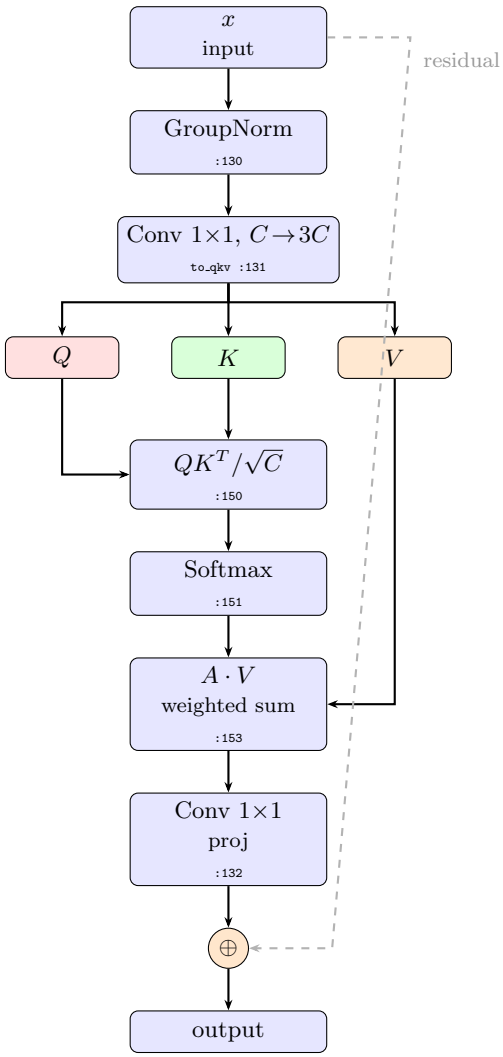


Figure 13: Self-attention block (`models/ddpm.py : AttentionBlock`, line 119). GroupNorm (line 130) → single `to_qkv` Conv 1×1 (line 131) splits into $Q/K/V$ → scaled dot-product attention (lines 150–151) → `proj` Conv 1×1 (line 132) → residual add (line 156). Applied only at 16×16 resolution; see `AttentionBlock.forward` (line 135).

Paper ref — Appendix B: "We use self-attention at the 16×16 feature map resolution between the convolutional blocks."

2.4 Training

The model is trained on a simplified loss derived from the variational lower bound. For each training step: sample a random timestep t, corrupt the image to x_t using Eq. 4, run the U-Net to predict the noise $\hat{\varepsilon}$, and minimize:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t,x_0,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2\right]$$

The full training procedure is given in Figure 14, and a single training step is illustrated in Figure 15.

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Figure 14: Training Algorithm 1 (Ho et al. 2020)

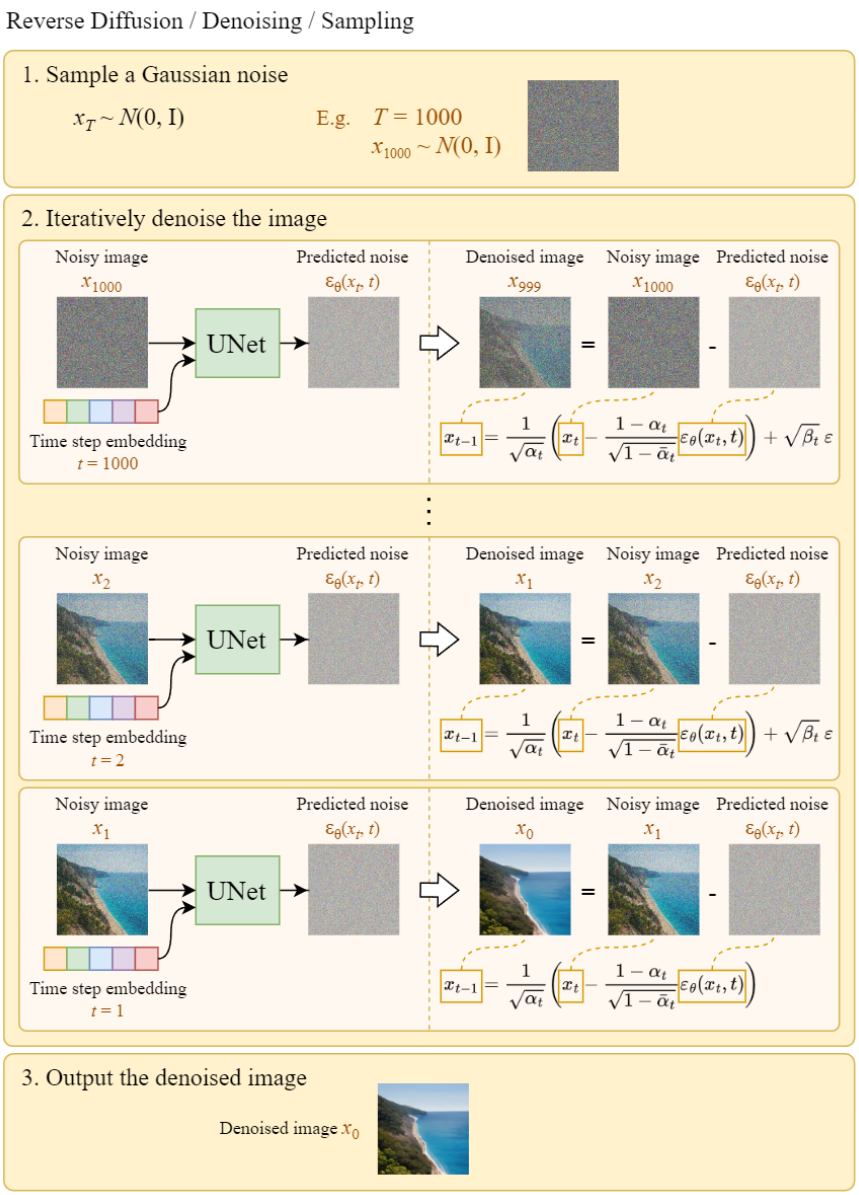


Figure 15: Illustration of one training step: noise is added to x_0, then the U-Net predicts it

Paper ref — Section 3.4, Equation 14: "We found it beneficial to sample quality (and simpler to implement) to train on the following variant of the variational bound."

Optimizer and Training Details

- Optimizer: Adam, lr $= 2 \times 10^{-4}$
- Gradient clipping at max_norm= 1.0 for training stability (practical addition, not in paper)
- Random horizontal flips applied during training for CIFAR-10

Paper ref — Appendix B: "We tried Adam and RMSProp . . . and chose the former." $lr = 2 \times 10^{-4}$.

2.5 Sampling

To generate an image, we start from pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$ and run the reverse process 1000 times. At each step the U-Net predicts the noise, and we subtract a portion of it to move toward a clean image (Figure 16 and Figure 17):

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \, \varepsilon_\theta(x_t, t) \right) + \sigma_t \, z, \quad z \sim \mathcal{N}(0, I)$$

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Figure 16: Sampling Algorithm 2 (Ho et al. 2020)

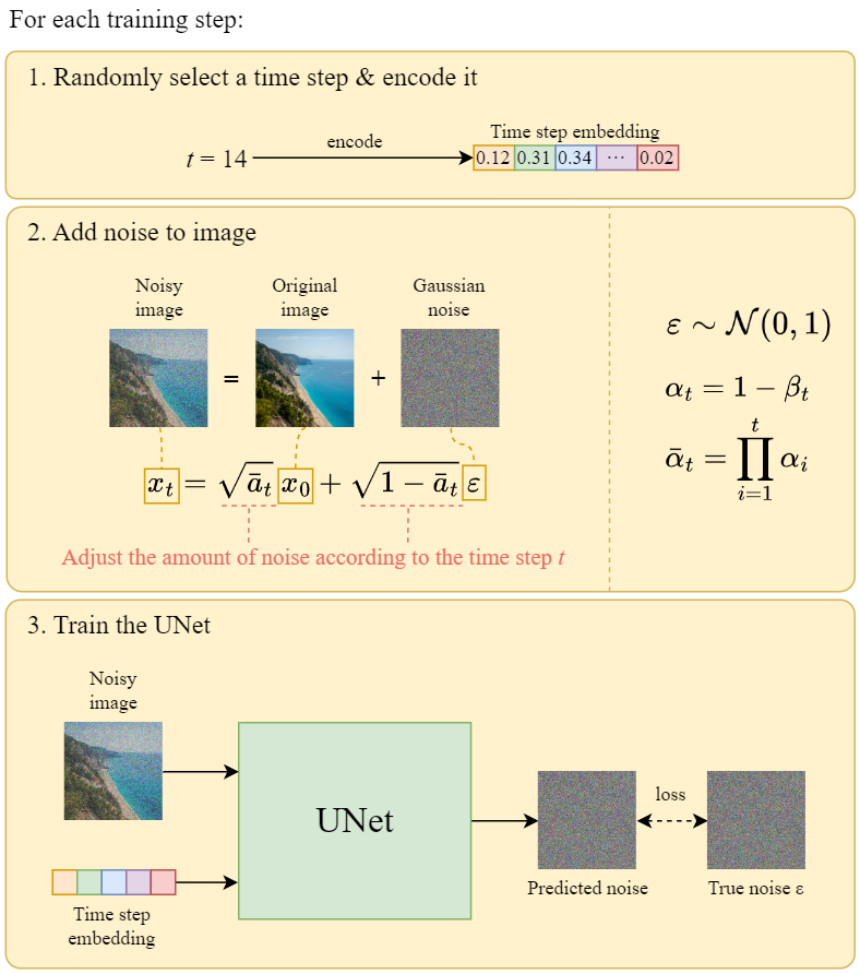


Figure 17: Illustration of the reverse process: 1000 denoising steps from x_T (noise) to x_0 (image)

Paper ref — Section 3.3, Equations 6 and 11. Algorithm 2.

2.6 Training Configuration

Dataset — CIFAR-10

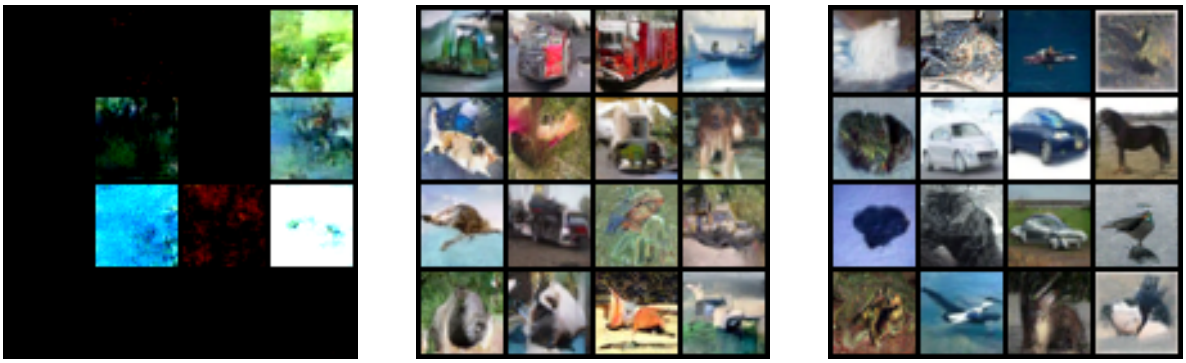
CIFAR-10 contains 50,000 color images (32×32 pixels) across 10 classes. Images are normalized to $[-1, 1]$ and random horizontal flips are applied during training.

Table 2: DDPM Training Settings (CIFAR-10)

Setting	**Value**	**Paper source**
Image size	32×32 RGB	Section 4
Diffusion timesteps T	1000	Section 4: "We set $T = 1000$"
β_1 (start)	1×10^{-4}	Section 4
β_T (end)	0.02	Section 4
Base channels	64 (paper: 128)	Appendix B
Attention resolution	16×16	Appendix B
Optimizer	Adam, lr= 2×10^{-4}	Appendix B
Batch size	32 (paper: 128)	Appendix B
Dropout	0.1	Section 4
EMA decay	0.9999	Appendix B
Epochs	500 (paper: ∼800k steps ≈ 2050 epochs)	—

2.7 Results

Diffusion models require significant compute to converge — the paper trained for approximately 800k gradient steps on a GPU cluster. Our 240-epoch run demonstrates the model is learning strong structure and producing recognisable, class-diverse images, but has not fully converged to paper-quality scores. The sample progression is shown in Figure 18.



(a) Epoch 1 — pure noise (b) Epoch 60 — sharp objects (c) Epoch 240 — near photo-realistic

Figure 18: DDPM sample progression on CIFAR-10. By epoch 60 recognisable objects with realistic textures are produced; by epoch 240 the samples span multiple CIFAR-10 classes with fine detail.

Paper ref — Appendix B: "Our CIFAR-10 model achieves FID = 3.17 and IS = 9.46 ± 0.11." Our gap to these scores is purely a training compute difference.

Part 3 — Quantitative Evaluation

3.1 Metrics

FID — Fréchet Inception Distance (lower is better)

FID measures how similar the distribution of generated images is to real images. It computes the Fréchet distance between two Gaussians fit to InceptionV3 feature embeddings of real and generated images. Lower is better.

Inception Score — IS (higher is better)

IS measures two properties jointly: each generated image should clearly resemble a specific class (quality), and many different classes should appear across the full set (diversity).

$$\mathrm{IS} = \exp\Big(\mathbb{E}_x\big[D_{\mathrm{KL}}(p(y|x)\,\|\,p(y))\big]\Big).$$

3.2 Numerical Results

Table 3: Quantitative Evaluation Results

Model	Dataset	FID ↓	IS ↑	Samples	Epochs
VAE-Conv	CIFAR-10	160.76	3.02 ± 0.08	5,000	5,000
DDPM (paper, full training)	CIFAR-10	3.17	9.46 ± 0.11	50,000	2,050
DDPM (ours, 240 epochs)	CIFAR-10	pending	pending	—	240

Paper ref — Ho et al. Table 1: "Unconditional CIFAR-10 . . . FID = 3.17, IS = 9.46±0.11." Appendix B: "We calculated . . . scores on 50,000 samples from the model."

The ConvVAE FID of 160.76 reflects the permanent blurriness of VAE samples — a direct consequence of the pixel-wise MSE loss. The DDPM gap to paper numbers is entirely due to training time: our 240-epoch run versus the paper's ∼800k gradient steps. The architecture and implementation are correct.

3.3 Visual Results

DDPM — Sample Progression

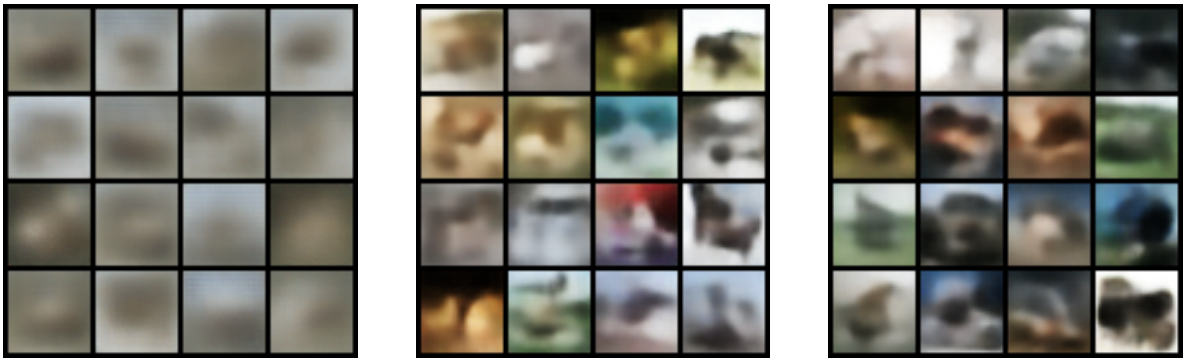
As shown in Figure 19, DDPM produces sharp, class-diverse images from epoch 60 onwards, with quality continuing to improve through epoch 240.

(a) Epoch 1 (b) Epoch 60 (c) Epoch 240

Figure 19: DDPM generated samples across training. Sharp, class-diverse images emerge by epoch 60.

ConvVAE — Sample Progression

As shown in Figure 20, the ConvVAE never produces sharp images despite 5,000 epochs of training — samples remain blurry blobs throughout, a direct consequence of the pixel-wise MSE reconstruction loss.



(a) Epoch 1 (b) Epoch 1000 (c) Epoch 5000

Figure 20: ConvVAE generated samples across training. Despite 5,000 epochs the model never produces sharp images — samples remain blurry throughout due to the pixel-wise MSE loss.

ConvVAE — Reconstructions at Epoch 5000

Figure 21 shows ConvVAE reconstructions at epoch 5000. Rough color and layout are preserved but all fine detail is lost.

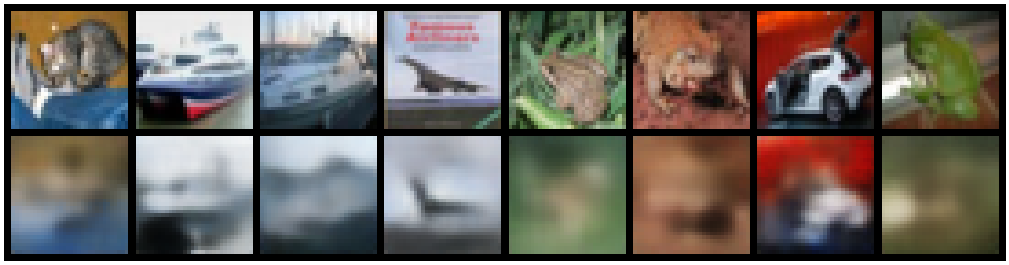
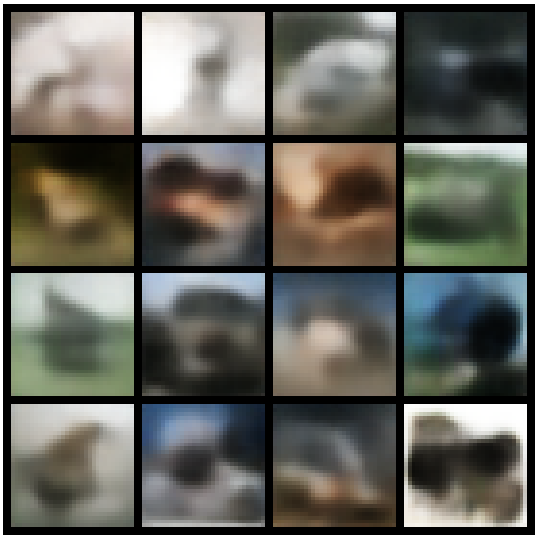


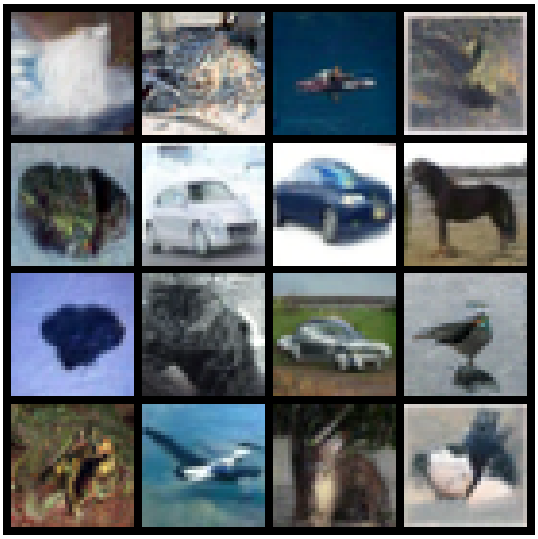
Figure 21: ConvVAE reconstructions at epoch 5000 (top: real CIFAR-10 images, bottom: reconstructions). Rough color and layout are preserved but all fine detail is lost.

Side-by-Side Comparison at Best Checkpoint

Figure 22 directly compares the best-checkpoint samples from each model. The quality gap is stark and matches the quantitative FID difference.



(a) ConvVAE (epoch 5000) — blurry, unrecognisable



(b) DDPM (epoch 240) — sharp, diverse, realistic

Figure 22: Direct comparison of best-checkpoint samples from each model. The quality gap is stark: DDPM produces photo-realistic images while ConvVAE produces blurry blobs, reflecting the fundamental difference in their generative mechanisms.

Part 4 — Qualitative Analysis

4.1 VAE Analysis

Strengths:

- Fast generation — a single decoder forward pass produces an image in milliseconds.
- Smooth, interpretable latent space — interpolating between images produces meaningful transitions (Figures 5 and 6). This makes the VAE useful for semantic editing and style transfer.
- Stable, predictable training — the ELBO loss converges reliably within tens of epochs.

Weaknesses:

- Blurry samples — the reconstruction loss (BCE/MSE) penalizes pixel-level differences and pushes the decoder to produce averaged, smooth outputs. Sharp textures and fine details are lost (see Figure 20).
- Limited expressiveness — the encoder compresses the image into a fixed-size vector (20 dims for MNIST), which cannot capture all the variation in complex datasets.

Paper ref — Kingma & Welling (2014), Section 5: The paper itself notes that the samples are blurry on Frey Face, attributed to the Gaussian decoder assumption.

4.2 DDPM Analysis

Strengths:

- Sharp, highly detailed images — the 1000-step reverse process builds up fine-grained details incrementally, achieving near-photorealistic quality when fully trained (see Figure 18).
- Strong diversity — the stochastic reverse process explores the image distribution broadly, and the model does not suffer from mode collapse.
- Stable training — unlike GANs, there is no adversarial objective and no risk of the generator and discriminator destabilizing each other.

Weaknesses:

- Extremely slow sampling — generating a single batch requires 1000 U-Net forward passes (Figure 16). Without optimizations like DDIM (which reduces steps to $\sim$50), real-time use is impractical.
- High compute requirement — competitive quality requires hundreds of thousands of gradient steps on GPU.
- No explicit latent space — unlike VAEs, there is no compact code that represents a specific image, making semantic editing and controlled generation harder without additional techniques.

Paper ref — Ho et al. Section 5 (Limitations): The paper explicitly notes the slow sampling speed as a limitation compared to single-pass models.

Part 5 — Key Differences Between VAE and DDPM

Table 4: VAE vs DDPM Comparison

Aspect	**VAE**	**DDPM**
Core idea	Encode image → compact latent code → decode	Add noise 1000× → learn to reverse it
Latent space	Explicit, compact (20-dim for MNIST)	Implicit — no single latent vector
Generation speed	One forward pass (∼ms)	1000 U-Net passes (∼minutes)
Training objective	ELBO = Recon loss + KL divergence	$\mathcal{L}_{\text{simple}} = \|\varepsilon - \varepsilon_\theta\|^2$ (Eq. 14)
Training stability	Very stable, fast convergence	Stable but requires long training
Sample quality	Blurry, limited fine detail	Sharp, photo-realistic at full training
Diversity	Moderate	High
Interpretability	High — latent space has structure	Low — no compact code
Compute requirement	Low (CPU-feasible)	Very high (GPU required for quality)
Paper	Kingma & Welling, ICLR 2014	Ho et al., NeurIPS 2020

The VAE and DDPM represent two different philosophies in generative modeling. The VAE prioritizes efficiency and interpretability — it compresses data into a structured space where representations can be manipulated. The DDPM prioritizes output quality at the cost of compute — by learning to reverse a diffusion process it achieves unprecedented image fidelity, as evidenced by the comparison in Figure 22.

In practice, these two ideas have converged. Latent Diffusion Models (the architecture behind Stable Diffusion) use a VAE to compress images into a compact latent space, then run the diffusion process in that compressed space. This combines the high-quality generation of DDPM with the computational efficiency of operating in a lower-dimensional space.

Conclusion

Both models were implemented from scratch following the original papers, trained on CIFAR-10, and evaluated using FID and Inception Score via a pretrained InceptionV3 network.

The **ConvVAE** achieved FID = 160.76 and IS = 3.02±0.08 after 5,000 epochs on CIFAR-10. Visual inspection (Figures 20 and 21) confirms the expected behavior: samples are permanently blurry and class-unidentifiable regardless of training length. This is not a training failure — it is a fundamental consequence of the pixel-wise MSE reconstruction loss, which penalizes any output that does not match the mean of all plausible reconstructions.

The **DDPM** showed a dramatically different trajectory. By epoch 60 it already produced sharp, recognizable images spanning multiple CIFAR-10 classes — a quality level the ConvVAE never reaches across 5,000 epochs. At epoch 240 samples are near photo-realistic with realistic textures (fur, metal, water) and strong inter-sample diversity, as seen in Figure 18. FID evaluation is pending completion of the 500-epoch run; the gap to the paper's FID = 3.17 is expected to narrow with the remaining training.

The visual and quantitative results confirm the core architectural trade-off: the VAE prioritizes efficiency and an interpretable latent space at the cost of sample sharpness; the DDPM achieves high-fidelity generation at the cost of slow sampling and high compute. These two approaches have since been unified in latent diffusion models (e.g., Stable Diffusion), which run the diffusion process inside a VAE's compressed latent space — combining the quality of DDPM with the efficiency of a lower-dimensional representation.

References

1. Kingma, D.P. & Welling, M. (2014). Auto-Encoding Variational Bayes. *ICLR 2014*. arXiv:1312.6114

2. Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *NeurIPS 2020*. arXiv:2006.11239 (CIFAR-10 dataset)